Tipos básicos

Tipos principales

int · números enteros
float · decimales
str · texto
bool · True o False
None · ausencia de valor

Código

```
entero = 42
flotante = 3.14
texto = "Python"
activo = True
nulo = None

print(type(texto))
```

```
# Ejemplos de nombres de variables válidos
mi_variable = 10
_contador = 0
nombreUsuario = "Carlos"
CONSTANTE = 3.14
```

```
# Ejemplos incorrectos (generarían error)
# 1variable = 5  # No puede comenzar con número
# mi-variable = 7  # No puede contener guiones
# class = "Python"  # No puede ser palabra reservada
```

Python permite la asignación múltiple de variables en una sola línea:

```
# Asignación múltiple
x, y, z = 10, "Hola", True
```

Conversión de tipos (casting)

Por qué importa

Python permite **convertir entre diferentes tipos de datos** mediante funciones de conversión:

```
# Conversión de tipos
numero_texto = "100"
numero = int(numero_texto)  # Convierte string a entero

flotante_a_entero = int(3.9)  # Trunca a 3, no redondea
entero_a_flotante = float(5)  # Convierte a 5.0

# Conversión a string
edad = 25
mensaje = "Tengo " + str(edad) + " años"
```

Código

```
edad_texto = "25"
edad = int(edad_texto)

precio = float("19.95")
mensaje = "Edad: " + str(edad)

print(mensaje)
```

input() devuelve texto
Para calcular hay que convertir
int() convierte a entero
float() convierte a decimal
str() convierte a texto

Operadores aritméticos

Los operadores aritméticos permiten realizar **operaciones matemáticas básicas.**

```
a = 10
b = 3

suma = a + b        # 13
resta = a - b       # 7
multiplicacion = a * b  # 30
division = a / b     # 3.3333... (siempre devuelve float)
division_entera = a // b  # 3 (descarta la parte decimal)
modulo = a % b       # 1 (resto de la división)
exponente = a ** b    # 1000 (a elevado a b)
```

Operadores de asignación

Los operadores de asignación permiten **asignar valores a variables,** a menudo combinados con otras operaciones.

```
x = 5      # Asignación básica

# Operadores compuestos
x += 3     # Equivale a: x = x + 3
x -= 2     # Equivale a: x = x - 2
x *= 4     # Equivale a: x = x * 4
x /= 2     # Equivale a: x = x / 2
x //= 2    # Equivale a: x = x // 2
x %= 3     # Equivale a: x = x % 3
x **= 2    # Equivale a: x = x ** 2
```

Operadores de comparación

Los operadores de comparación **evalúan condiciones y devuelven valores booleanos** (True o False).

```
a = 10
b = 5

igual = a == b        # False
diferente = a != b    # True
mayor = a > b         # True
menor = a < b         # False
mayor_igual = a >= b  # True
menor_igual = a <= b  # False
```

Operadores lógicos

Los operadores lógicos permiten **combinar expresiones booleanas.**

```
x = True
y = False

resultado_and = x and y  # False (ambos deben ser True)
resultado_or = x or y    # True (al menos uno debe ser True)
resultado_not = not x    # False (invierte el valor)
```

Operadores de identidad

Los operadores de identidad **comprueban si dos variables hacen referencia al mismo objeto** en memoria.

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a

# True (a y c apuntan al mismo objeto)
es_mismo_objeto = a is c
# True (a y b son objetos distintos)
no_es_mismo_objeto = a is not b
```

Operadores de pertenencia

Los operadores de pertenencia verifican **si un valor está presente** en una secuencia.

```
lista = [1, 2, 3, 4, 5]
texto = "Python"

contiene = 3 in lista       # True
no_contiene = 6 in lista    # False
letra_en_texto = "y" in texto  # True
```

Cadenas de texto

Las cadenas de texto en Python son **secuencias inmutables de caracteres** y se pueden definir con **comillas simples o dobles.**

```
nombre = 'Ana'
mensaje = "Hola, mundo"

# Cadenas multilínea con triple comilla
descripcion = """Este es un texto
que ocupa varias
líneas"""
```

Cadenas de texto

Operaciones con cadenas

Concatenación:

```
saludo = "Hola" + " " + "Python"  # "Hola Python"
```

Repetición:

```
repeticion = "Python" * 3  # "PythonPythonPython"
```

Acceso a caracteres o indexación:

```
primera_letra = saludo[0]  # "H"
ultima_letra = saludo[-1]  # "n"
```

Slicing o subcadenas:

```
subcadena = saludo[0:4]  # "Hola"
```

Las cadenas incluyen **métodos útiles** para su manipulación:

```
texto = "  Python es genial  "
```

Cadenas de texto

Métodos

Métodos comunes de *strings:*

```
mayusculas = texto.upper()       # "  PYTHON ES GENIAL  "
minusculas = texto.lower()       # "  python es genial  "
sin_espacios = texto.strip()     # "Python es genial"
reemplazo = texto.replace("genial", "increíble")  # "  Python es increíble  "
```

Verificación:

```
empieza_con = texto.strip().startswith("Python")  # True
termina_con = texto.strip().endswith("genial")    # True
contiene = "es" in texto  # True
```

Cadenas de texto

Formato de cadenas

Python ofrece varias formas de formatear cadenas **para incluir valores de variables:**

```
nombre = "Ana"
edad = 25

# Método format()
mensaje1 = "Me llamo {} y tengo {} años".format(nombre, edad)

# F-strings (desde Python 3.6)
mensaje2 = f"Me llamo {nombre} y tengo {edad} años"

# Formateo con especificadores
pi = 3.14159
mensaje3 = f"El valor de pi es {pi:.2f}"  # "El valor de pi es 3.14"
```

Cadenas de texto

Constantes y convenciones de nomenclatura

Aunque Python no tiene constantes verdaderas, existe la **convención** de usar nombres en mayúsculas para valores que no deberían cambiar:

```
PI = 3.14159
GRAVEDAD = 9.8
MAX_USUARIOS = 100

# Convención para variables "privadas"
_contador_interno = 0  # Indica que no debería accederse directamente
```

Python utiliza snake_case como convención para nombrar **variables y funciones,** mientras que para las **clases** se usa PascalCase:

```
nombre_usuario = "carlos123"  # Variable en snake_case
contador_total = 0         # Variable en snake_case

class UsuarioRegistrado:    # Clase en PascalCase
   pass
```

Cadenas de texto

Operaciones habituales

Concatenar con +
Repetir con *
Acceder por índice
Usar métodos como upper() o strip()
Formatear con f-strings

Código

```
nombre = "Ana"
edad = 25

saludo = f"Hola, {nombre}"
print(saludo)

texto = "  Python  "
print(texto.strip())
print(texto.upper())
```

Entrada por teclado

Actividad guiada

Pedir dos números
Convertirlos a float
Mostrar suma, resta y multiplicación
Evitar introducir código nuevo innecesario

Código

```
n1 = float(input("Primer número: "))
n2 = float(input("Segundo número: "))

print("Suma:", n1 + n2)
print("Resta:", n1 - n2)
print("Producto:", n1 * n2)
```

Condicionales

Estructura

if evalúa una condición
elif añade casos intermedios
else cubre el resto
La indentación define el bloque

Código

```
edad = 18

if edad < 18:
    print("Menor de edad")
elif edad == 18:
    print("Justo 18")
else:
    print("Mayor de edad")
```

Comparaciones y lógica

Operadores

== igual
!= diferente
> mayor, < menor
and exige dos condiciones ciertas
or exige al menos una
not invierte el valor

Código

```
nota = 7
asistencia = 85

if nota >= 5 and asistencia >= 80:
    print("Aprobado")
else:
    print("Revisar")
```

Bucle for

Cuándo usarlo

Cuando conocemos la colección
Cuando recorremos una lista
Cuando usamos range()
Cuando interesa procesar elemento a elemento

Código

```
frutas = ["manzana", "pera", "uva"]

for fruta in frutas:
    print(fruta)

for i in range(5):
    print(i)
```

Bucle while

Cuándo usarlo

Cuando repetimos hasta que cambie una condición
En menús por consola
Conviene modificar la variable de control
Evitar bucles infinitos

Código

```
opcion = ""

while opcion != "salir":
    opcion = input("Opción: ")
    print("Elegida:", opcion)
```

break y continue

Control del bucle

break termina el bucle
continue salta a la siguiente vuelta
Útiles, pero no sustituyen un buen diseño
Mantener el código legible

Código

```
for numero in range(1, 11):
    if numero % 2 == 0:
        continue
    if numero > 7:
        break
    print(numero)
```

Listas

Idea clave

Colección ordenada
Mutable
Acceso por índice
Métodos: append, remove, sort, pop

Código

```
notas = [7, 5, 9]
notas.append(8)
notas[1] = 6

print(notas)
print(notas[0])
print(len(notas))
```

Tuplas y conjuntos

Comparación

Tupla: ordenada e inmutable
Set: elementos únicos
Set elimina duplicados
Elegir según el problema

Código

```
coordenada = (10, 20)
print(coordenada[0])

notas = [7, 7, 8, 9, 8]
unicas = set(notas)
print(unicas)
```

Diccionarios

Idea clave

Guardan pares clave-valor
Acceso rápido por clave
get() evita errores si falta la clave
Muy útiles para modelar entidades

Código

```
alumno = {
    "nombre": "Ana",
    "curso": "2º DAM",
    "nota": 8
}

print(alumno["nombre"])
print(alumno.get("email", "Sin email"))
```

Funciones

Por qué usarlas

Evitan repetir código
Dividen el problema en partes
Hacen el programa más legible
Facilitan probar cada pieza

Código

```
def saludar(nombre):
    print(f"Hola, {nombre}“)

saludar("Ana")
saludar("Carlos")
```

Parámetros y retorno

Idea clave

Los parámetros son datos de entrada
return devuelve un resultado
Sin return explícito se devuelve None
Una función debe hacer una tarea clara

Código

```
def cuadrado(x):
    return x * x

resultado = cuadrado(4)
print(resultado)
```

Funciones con listas

```
def calcular_promedio(notas):
    if len(notas) == 0:
        return 0
    return sum(notas) / len(notas)

def clasificar_nota(nota):
    if nota < 5:
        return "Suspenso"
    if nota < 7:
        return "Bien"
    if nota < 9:
        return "Notable"
    return "Sobresaliente"
```

Excepciones

Problema típico

El usuario puede escribir mal
Un archivo puede no existir
Una división puede fallar
El programa debe responder sin romperse

Código

```
try:
    numero = int(input("Número: "))
    print(10 / numero)
except ValueError:
    print("Número no válido")
except ZeroDivisionError:
    print("No se puede dividir entre cero")
```

finally

Uso

Se ejecuta ocurra o no ocurra error
Útil para cerrar recursos
También sirve para mostrar cierre del proceso
No sustituye a except

Código

```
try:
    archivo = open("datos.txt")
    print(archivo.read())
except FileNotFoundError:
    print("Archivo no encontrado")
finally:
    print("Fin de la operación")
```

Módulos propios

Idea clave

Un módulo es un archivo .py
Permite reutilizar funciones
Se importa con import o from
Ayuda a ordenar proyectos pequeños

Código

```
# utilidades.py
def sumar(a, b):
    return a + b

# main.py
from utilidades import sumar
print(sumar(2, 3))
```

Archivos de texto

Patrón recomendado

Usar with open(...)
Indicar encoding utf-8
Escribir con write()
Leer línea a línea cuando sea sencillo

Código

```
with open("notas.txt", "w", encoding="utf-8") as f:
    f.write("Ana;7\n")

with open("notas.txt", "r", encoding="utf-8") as f:
    for linea in f:
        print(linea.strip())
```